MIPT & YSDA · AUTUMN 2026

Deep Generative Models

Lecture 1

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Outline

01 Introduction and Logistics

02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Outline

01 **Introduction and Logistics**

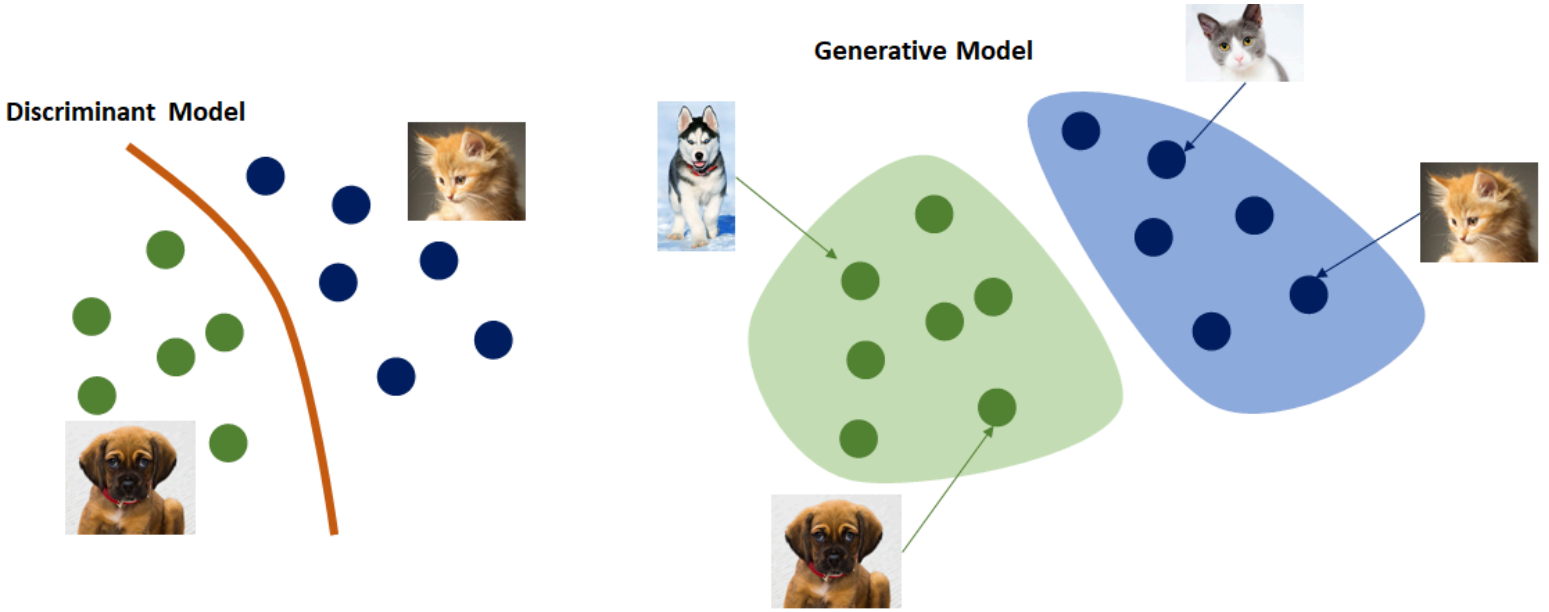
02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Generative vs Discriminative Models



- **Discriminative model:** learns a decision boundary $p(\mathbf{y}|\mathbf{x})$.
- **Generative model:** learns the data distribution $p_{\text{data}}(\mathbf{x})$ (optionally $p_{\text{data}}(\mathbf{x}, \mathbf{y})$) and supports **sampling (generation)**.

Course Team

Roman Isachenko

lecturer

PhD in Phys. & Math.

CV Engineer
at Yandex

`@roman_isachenko`

Matvey Morozov

seminarist

MIPT alumnus (2020)

Senior CV Engineer
at Gradient & Persona

`@morozov_ma`

Grigory Ksenofontov

teaching assistant

MIPT alumnus (2023)

PhD student
at Skoltech

`@gregkseno`

About the Course

Theoretical foundations & practical applications of **Deep Generative Models**. Primary focus: Computer Vision, but the principles are universal across modern AI.

Six major pillars

- **Likelihood-based:** Autoregressive, Normalizing Flows.
- **Latent Variable:** VAE, VQ-VAE.
- **Adversarial:** GAN, WGAN, model quality evaluation.
- **Score-based & Diffusion:** score matching, NCSN, DDPM, guidance.
- **Continuous-Time:** CNF, SDE/ODE, Flow Matching.
- **Discrete Diffusion:** absorbing, masked diffusion LMs.

Course Structure and Grading

Structure

- **14** lectures + **14** seminars (one per week)
- **6** homework assignments
- cozy oral exam

Game rules

	Points
Homeworks (6×15 pts)	**90**
Oral exam	**30**
Maximum	**120**

Final grade

$$\text{grade} = \min\left(\left\lfloor \frac{\#\text{points}}{10} \right\rfloor, 10\right)$$

Lecture Schedule

1. Intro & Logistics; AR models (ImageGPT)
2. Normalizing Flows (NF); LVM intro
3. ELBO, Reparametrization Trick, VAE
4. ELBO Surgery; Discrete VAE; VQ-VAE; Likelihood-free Learning
5. GAN; Wasserstein Distance; WGAN; Model Quality Evaluation
6. Langevin Dynamics; Score Matching; NCSN
7. Gaussian Diffusion; Diffusion ELBO
8. DDPM; Classifier(-Free) Guidance
9. Continuous-Time NF; Continuity Equation; SDE basics
10. Probability Flow ODE; Reverse SDE; Flow Matching
11. Conditional Flow Matching (CFM)
12. Two-Sided Conditioning; FM $\leftrightarrow$ Score link; Discrete Diffusion
13. Discrete Diffusion: Reverse Process; Absorbing Diffusion
14. Course Overview and Recap

Prerequisites

What you should know

- **Math:** probability theory, statistics, multivariate calculus.
- **ML:** fundamentals of machine learning and deep learning.
- **Tools:** PyTorch / basic GPU workflow is a plus.
- **Optional:** information theory basics (entropy, KL divergence).
- **Optional:** familiarity with numerical ODE concepts at a “black-box usage” level.

Key points

- The course is **mathematically heavy**.
- The course is **constantly evolving**.
- Any feedback — especially *negative* — is very welcome!

Repository: github.com/r-isachenko/2026-DGM-MIPT-YSDA-course

Outline

01 Introduction and Logistics

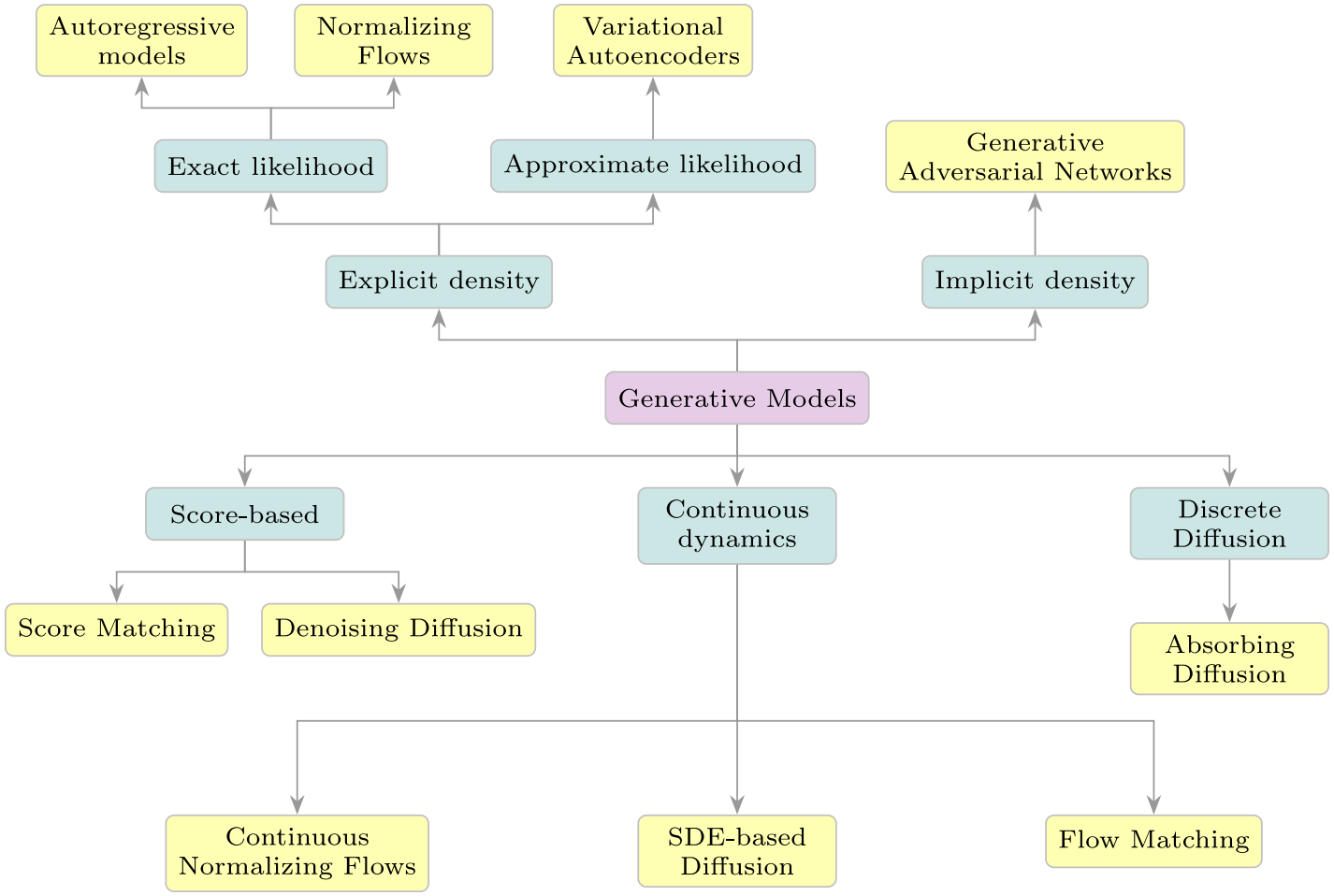
02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Generative Models Taxonomy



VAE — The First Scalable Approach for Image Generation

Kingma D.P., Welling M. Auto-Encoding Variational Bayes, 2013

DCGAN — The First Convolutional GAN for Image Generation

Radford A., Metz L., Chintala S. Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks, 2015

StyleGAN — High-Quality Face Generation

Language Modeling at Scale

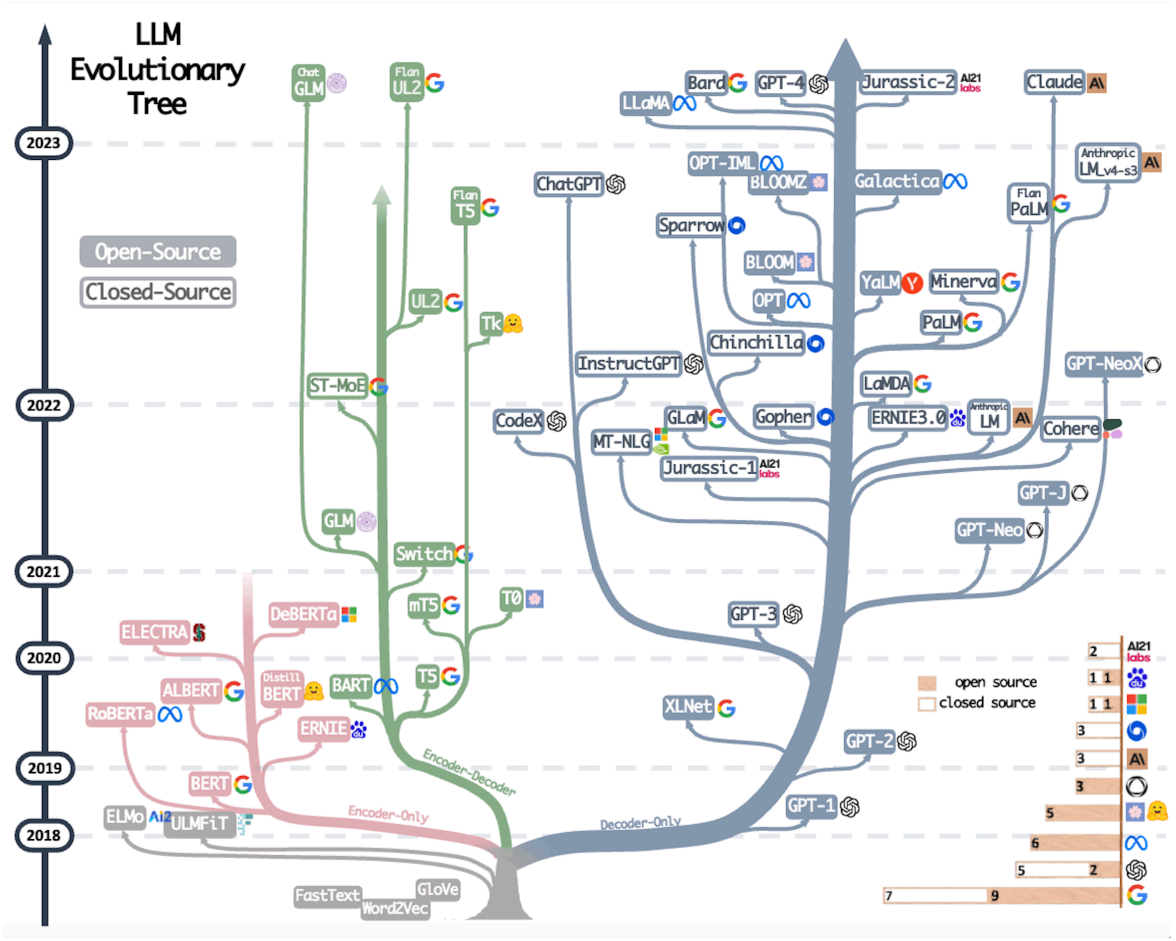


Image credit: https://blog.biocomm.ai/2023/05/14/open-source-proliferation-llm-evolutionary-tree/

Language Models: 2023–2026

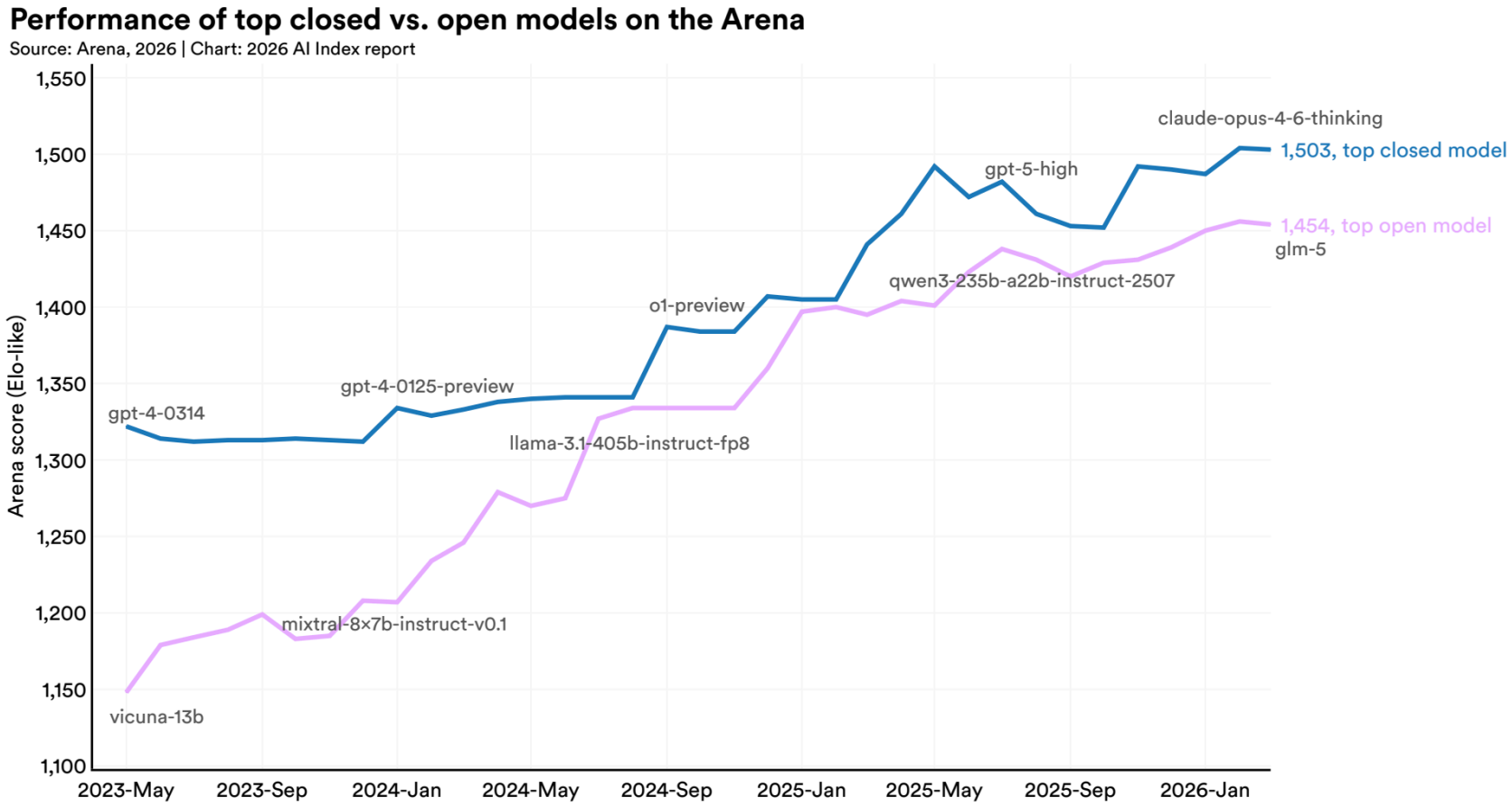


Figure 2.1.2[2]

Stanford HAI. AI Index Report 2026, Figure 2.1.2, p. 77. Source: Arena historical leaderboard, March 2026.
Arena scores summarize human preferences in pairwise comparisons.

Diffusion Models — A Breakthrough in Image Generation

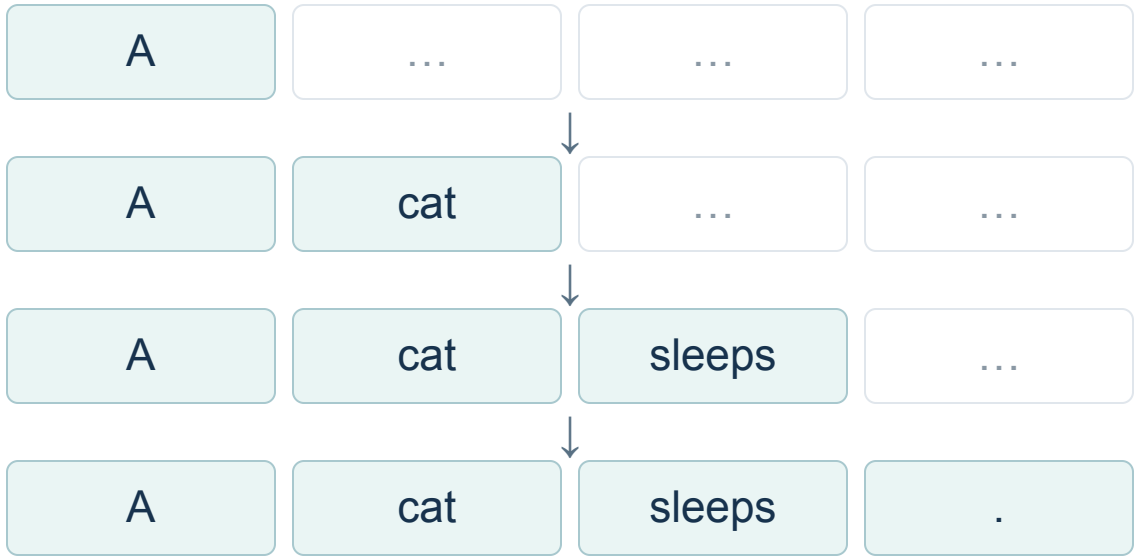
DDPM (2020) → Diffusion beats GANs (2021) → Midjourney (2022)

Ho J. et al. Denoising Diffusion Probabilistic Models, 2020 · Dhariwal P., Nichol A. Diffusion Models Beat GANs on Image Synthesis, 2021
Midjourney gallery · Midjourney model history (2022)

Diffusion Language Models — LLaDA (2025)

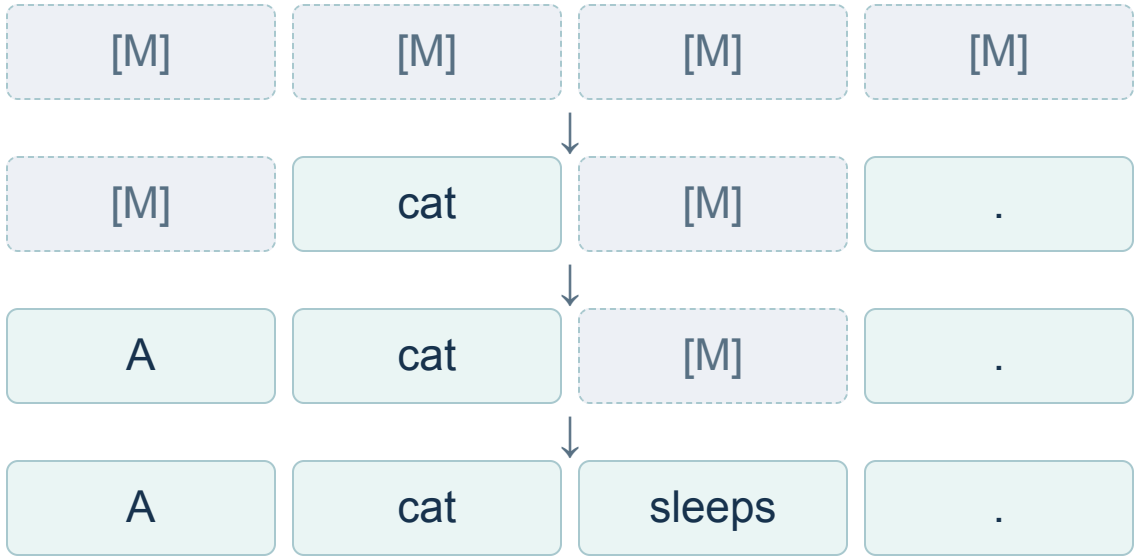
Language generation can use **iterative unmasking**.

Autoregressive



Append tokens from left to right.

Masked diffusion



Restore tokens across the sequence.

Nie S. et al. Large Language Diffusion Models, 2025. · LLaDA project page
Schematic selected sampling steps; [M] denotes a masked token. Example words stand in for tokens.

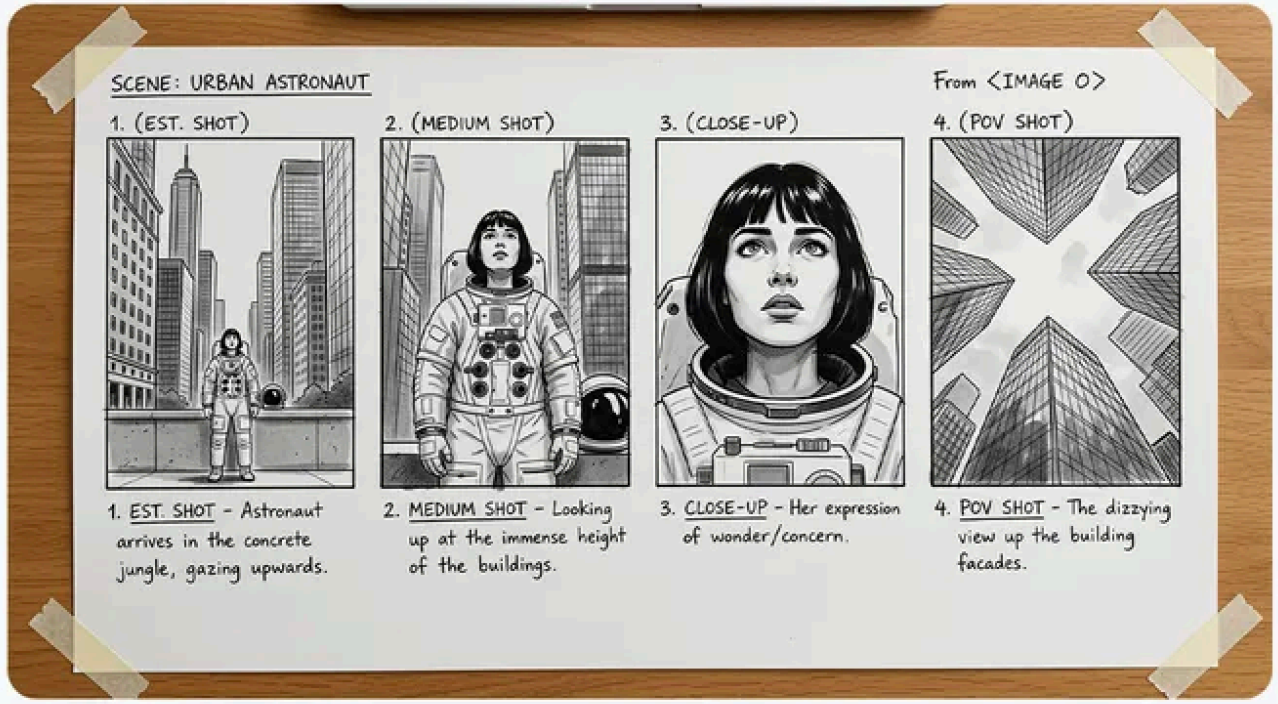
Sora — Video Generation (2024)

Image credit: https://openai.com/index/sora

Nano Banana Pro (2025)

Prompt: Create a storyboard for this scene

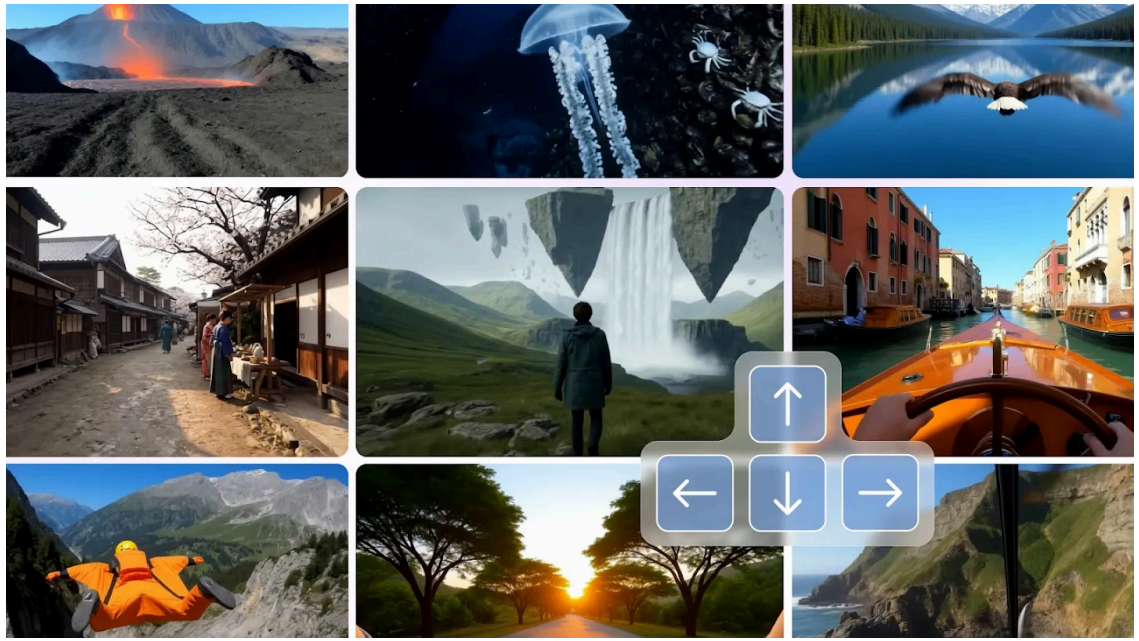
Input image



Output image

Image credit: https://blog.google/technology/ai/nano-banana-pro/

Genie 3 — Interactive World Generation (2025)

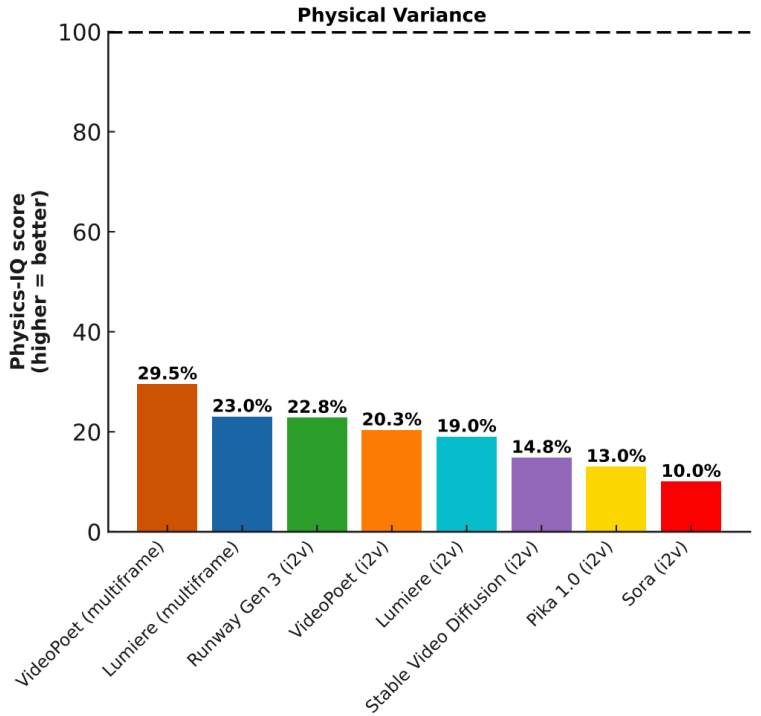


- Start from a text prompt.
- Navigate the generated environment.
- Future frames respond to your actions.

World model: predicts how observations evolve under actions.

Open Problems in Generative Models

- Video generation
- 3D scene generation
- Understanding of physical processes
- Multimodal end-to-end models



Motamed S. et al. Do generative video models understand physical principles?, 2025

Outline

01 Introduction and Logistics

02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Outline

01 Introduction and Logistics

02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Course Tricks I

We omit density subscripts: $p(\mathbf{x}) \equiv p_{\mathbf{x}}(\mathbf{x})$.

Log-Derivative Trick (LGT)

Given a differentiable function $p : \mathbb{R}^m \to \mathbb{R}$ (usually density function),

$$\nabla \log p(\mathbf{x}) = \frac{1}{p(\mathbf{x})} \cdot \nabla p(\mathbf{x}).$$

Jensen's Inequality (JI)

If $\mathbf{x} \in \mathbb{R}^m$ is a continuous random variable with density $p(\mathbf{x})$ and $f : \mathbb{R}^m \to \mathbb{R}$ is convex, then

$$\mathbb{E}[f(\mathbf{x})] \geq f(\mathbb{E}[\mathbf{x}]).$$

Monte Carlo Estimation (MC)

Let $\mathbf{x} \in \mathbb{R}^m$ be a continuous random variable with density $p(\mathbf{x})$, and $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^d$ be any vector-valued function. Then,

$$\mathbb{E}_{p(\mathbf{x})}\mathbf{f}(\mathbf{x}) = \int p(\mathbf{x})\mathbf{f}(\mathbf{x})d\mathbf{x} \approx \frac{1}{n}\sum_{i=1}^{n}\mathbf{f}(\mathbf{x}_i), \quad \text{where } \mathbf{x}_i \sim p(\mathbf{x}).$$

Course Tricks II

Law of the Unconscious Statistician (LOTUS)

Let $\mathbf{x} \sim p(\mathbf{x})$, $\mathbf{y} = \mathbf{f}(\mathbf{x})$, and $\mathbf{f}, \mathbf{g}$ be measurable with $\mathbb{E}\|\mathbf{g}(\mathbf{y})\| < \infty$. Then

$$\mathbb{E}[\mathbf{g}(\mathbf{y})] = \mathbb{E}_{p(\mathbf{x})}\mathbf{g}(\mathbf{f}(\mathbf{x})) = \int p(\mathbf{x})\mathbf{g}(\mathbf{f}(\mathbf{x}))d\mathbf{x}.$$

Change of Variables Theorem (CoV)

Let $\mathbf{x} \in \mathbb{R}^m$ be a random vector with density $p(\mathbf{x})$, and let $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^m$ be a C^1-diffeomorphism ($\mathbf{f}$ and $\mathbf{f}^{-1}$ are continuously differentiable mappings). If $\mathbf{z} = \mathbf{f}(\mathbf{x})$, then

$$p(\mathbf{x}) = p(\mathbf{z})|\det(\mathbf{J}_{\mathbf{f}})| = p(\mathbf{z})\left|\det\left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}}\right)\right| = p(\mathbf{f}(\mathbf{x}))\left|\det\left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}}\right)\right|$$

$$p(\mathbf{z}) = p(\mathbf{x})|\det(\mathbf{J}_{\mathbf{f}^{-1}})| = p(\mathbf{x})\left|\det\left(\frac{\partial \mathbf{x}}{\partial \mathbf{z}}\right)\right| = p(\mathbf{f}^{-1}(\mathbf{z}))\left|\det\left(\frac{\partial \mathbf{f}^{-1}(\mathbf{z})}{\partial \mathbf{z}}\right)\right|$$

Course Tricks III

Optimal MSE Prediction

Let $(\mathbf{x}, \mathbf{y})$ be jointly distributed with density $p(\mathbf{x}, \mathbf{y})$. The optimal predictor under mean squared error is the conditional expectation:

$$\mathbf{f}^*(\mathbf{x}) = \arg\min_{\mathbf{f}} \mathbb{E}_{p(\mathbf{x},\mathbf{y})} \|\mathbf{y} - \mathbf{f}(\mathbf{x})\|^2 = \mathbb{E}_{p(\mathbf{y}|\mathbf{x})}[\mathbf{y}].$$

Dirac Delta Function

Any deterministic variable $\mathbf{x}_0$ can be interpreted as a random variable with density $p(\mathbf{x}) = \delta(\mathbf{x} - \mathbf{x}_0)$.

$$\delta(\mathbf{x}) = \begin{cases} +\infty, & \mathbf{x} = 0 \\ 0, & \mathbf{x} \neq 0 \end{cases} \qquad \int \delta(\mathbf{x}) d\mathbf{x} = 1$$

$$\mathbb{E}_{p(\mathbf{x})} \mathbf{f}(\mathbf{x}) = \int \delta(\mathbf{x} - \mathbf{x}_0) \mathbf{f}(\mathbf{x}) d\mathbf{x} = \mathbf{f}(\mathbf{x}_0)$$

Outline

01 Introduction and Logistics

02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Problem Statement

We're given a **finite** number of i.i.d. samples $\{\mathbf{x}_i\}_{i=1}^n \subset \mathbb{R}^m$ drawn from an **unknown** distribution $p_{\text{data}}(\mathbf{x})$.

Objective

Learn a model $p(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ from data (**density estimation**). Two model operations:

- **Sampling (generation):** sample a new object $\mathbf{x} \sim p(\mathbf{x})$.
- **Density evaluation:** compute $p(\mathbf{x})$ for a given object $\mathbf{x}$.

Challenge

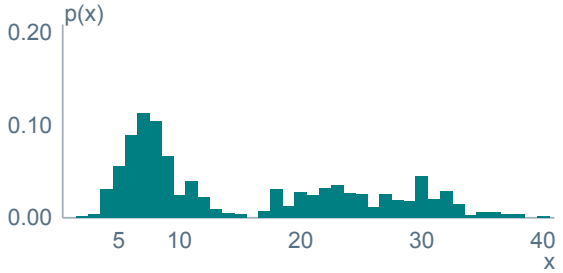
The data is high-dimensional and complex. For example, image datasets live in $\mathbb{R}^{\text{width} \times \text{height} \times \text{channels}}$. The curse of dimensionality makes accurately estimating $p_{\text{data}}(\mathbf{x})$ infeasible.

Histogram as a Generative Model

Assume $x \sim \mathrm{Cat}(\boldsymbol{\pi})$. The histogram model is fully characterized by

$$\hat{\pi}_k = \hat{\pi}(x = k) = \frac{\sum_{i=1}^{n} [x_i = k]}{n}.$$

Curse of dimensionality: The number of bins rises exponentially.



n = 1000 observations

MNIST example: 28×28 grayscale images, with each image $\mathbf{x} = (x_1, \ldots, x_{784})$, $x_i \in \{0, 1\}$:

$$p_{\text{data}}(\mathbf{x}) = p(x_1) \cdot p(x_2|x_1) \cdot \cdots \cdot p(x_m|x_{m-1}, \ldots, x_1).$$

A complete histogram would require $2^{28 \times 28} - 1$ parameters for $p_{\text{data}}(\mathbf{x})$.

Question: How many parameters are required in these cases?

$$p_{\text{data}}(\mathbf{x}) = p(x_1) \cdot p(x_2) \cdot \cdots \cdot p(x_m);$$

$$p_{\text{data}}(\mathbf{x}) = p(x_1) \cdot p(x_2|x_1) \cdot \cdots \cdot p(x_m|x_{m-1}).$$

Conditional Models

In practice, we're typically interested in learning conditional models (sampling from conditional distribution $p_{\text{data}}(\mathbf{x}|\mathbf{y})$).

Condition $\mathbf{y}$	Output $\mathbf{x}$	Model
$\emptyset$	image	unconditional image model
class label	image	class-conditional image model
text prompt	image	text-to-image model
image	image	image-to-image model
image	text	image-to-text (image captioning) model
English text	Russian text	sequence-to-sequence model (machine translation)
sound	text	speech-to-text (automatic speech recognition) model
text	sound	text-to-speech model

Outline

01 Introduction and Logistics

02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Divergences

- Let us fix a probabilistic model $p_{\boldsymbol{\theta}}(\mathbf{x})$ from a parametric family of distributions $\{p(\mathbf{x}|\boldsymbol{\theta}) \mid \boldsymbol{\theta} \in \boldsymbol{\Theta}\}$.
- Instead of searching among all possible distributions for the true $p_{\text{data}}(\mathbf{x})$, we seek a functional approximation $p_{\boldsymbol{\theta}}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$.

What is a Divergence?

Let $\mathcal{P}$ be the set of all probability distributions. A mapping $D : \mathcal{P} \times \mathcal{P} \to \mathbb{R}$ is called a **divergence** if

- $D(\pi \| p) \geq 0$ for all $\pi, p \in \mathcal{P}$
- $D(\pi \| p) = 0$ if and only if $\pi \equiv p$

Divergence Minimization Problem

$$\min_{\boldsymbol{\theta}} D(p_{\text{data}} \| p_{\boldsymbol{\theta}})$$

where $p_{\text{data}}(\mathbf{x})$ is the true data distribution and $p_{\boldsymbol{\theta}}(\mathbf{x})$ is the model distribution.

Forward KL vs Reverse KL (Kullback-Leibler Divergence)

Forward KL

$$D_{\mathrm{KL}}(p_{\text{data}} \| p_{\boldsymbol{\theta}}) = \int p_{\text{data}}(\mathbf{x}) \log \frac{p_{\text{data}}(\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{x})} d\mathbf{x} \to \min_{\boldsymbol{\theta}}$$

Reverse KL

$$D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \| p_{\text{data}}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}) \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} d\mathbf{x} \to \min_{\boldsymbol{\theta}}$$

What's the practical distinction between these two objectives?

Maximum Likelihood Estimation (MLE)

Let $\{\mathbf{x}_i\}_{i=1}^n$ be i.i.d. observed samples.

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{n} p_{\boldsymbol{\theta}}(\mathbf{x}_i) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(\mathbf{x}_i).$$

Forward KL vs Reverse KL: MLE as Forward KL

Forward KL

$$
\begin{aligned}
D_{\mathrm{KL}}(p_{\text{data}} \| p_{\boldsymbol{\theta}}) &= \int p_{\text{data}}(\mathbf{x}) \log \frac{p_{\text{data}}(\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{x})} d\mathbf{x} \\
&= \int p_{\text{data}}(\mathbf{x}) \log p_{\text{data}}(\mathbf{x}) d\mathbf{x} - \int p_{\text{data}}(\mathbf{x}) \log p_{\boldsymbol{\theta}}(\mathbf{x}) d\mathbf{x} \\
&= -\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x})] + \text{const} \\
&\approx -\frac{1}{n} \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(\mathbf{x}_i) + \text{const} \to \min_{\boldsymbol{\theta}}.
\end{aligned}
$$

> **Maximum likelihood estimation** is thus equivalent to minimizing a Monte Carlo estimate of the **forward KL divergence**.

Reverse KL

$$
\begin{aligned}
D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \| p_{\text{data}}) &= \int p_{\boldsymbol{\theta}}(\mathbf{x}) \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} d\mathbf{x} \\
&= \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{x})} \left[\log p_{\boldsymbol{\theta}}(\mathbf{x}) - \log p_{\text{data}}(\mathbf{x})\right] \to \min_{\boldsymbol{\theta}}
\end{aligned}
$$

Outline

01 Introduction and Logistics

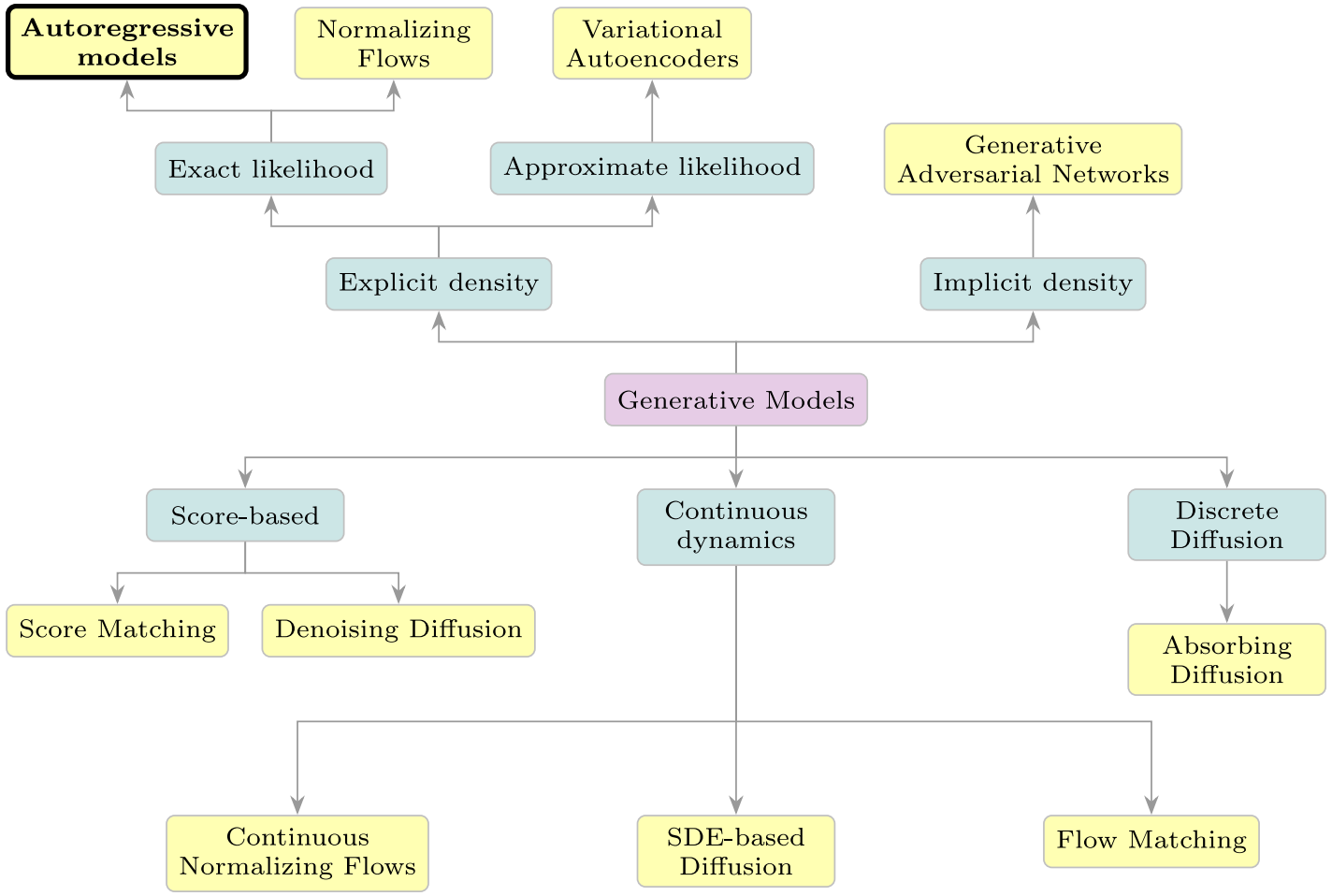
02 Generative Models Overview

03 Generative Modeling Framework

Course Tricks
Problem Statement
Divergence Minimization Framework

04 Autoregressive Models (ImageGPT)

Generative Models Taxonomy



Autoregressive Modeling

MLE Problem

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{n} p_{\boldsymbol{\theta}}(\mathbf{x}_i) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

- This maximization is typically solved via gradient-based optimization.
- Thus, efficient computation of both $\log p_{\boldsymbol{\theta}}(\mathbf{x})$ and its gradient $\frac{\partial \log p_{\boldsymbol{\theta}}(\mathbf{x})}{\partial \boldsymbol{\theta}}$ is crucial.

Likelihood as a Product of Conditionals

For $\mathbf{x} = (x_1, \ldots, x_m)$, $\mathbf{x}_{1:j} = (x_1, \ldots, x_j)$,

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{j=1}^{m} p_{\boldsymbol{\theta}}(x_j | \mathbf{x}_{1:j-1}); \quad \log p_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=1}^{m} \log p_{\boldsymbol{\theta}}(x_j | \mathbf{x}_{1:j-1})$$

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \left[\sum_{j=1}^{m} \log p_{\boldsymbol{\theta}}(x_{ij} | \mathbf{x}_{i,1:j-1}) \right]$$

Autoregressive Models

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=1}^{m} \log p_{\boldsymbol{\theta}}(x_j | \mathbf{x}_{1:j-1})$$

Sampling (Ancestral)

1. Sample $\hat{x}_1 \sim p_{\boldsymbol{\theta}}(x_1)$, $\hat{x}_2 \sim p_{\boldsymbol{\theta}}(x_2 | \hat{x}_1)$, ..., $\hat{x}_m \sim p_{\boldsymbol{\theta}}(x_m | \hat{\mathbf{x}}_{1:m-1})$.
2. Return $\hat{\mathbf{x}} = (\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_m)$.

- Each conditional $p_{\boldsymbol{\theta}}(x_j | \mathbf{x}_{1:j-1})$ can be modeled using a neural network.
- Modeling all conditionals separately isn't feasible. To address this, we share parameters across all conditionals.

Sampling vs Density Evaluation

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{j=1}^{m} p_{\boldsymbol{\theta}}(x_j \mid \mathbf{x}_{1:j-1}), \qquad \log p_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=1}^{m} \log p_{\boldsymbol{\theta}}(x_j \mid \mathbf{x}_{1:j-1}).$$

Raster order · white = 0 · dark = 1
Gray = not sampled

One model, two operations

Sampling: sample a pixel from its conditional distribution; append it to the context.

Density evaluation: use the observed prefix; multiply the probabilities of the observed pixels.

pixel 1	pixel 2	pixel 3	pixel 4	pixel 5	pixel 6	pixel 7	pixel 8	pixel 9
0.563	**0.722**	**1.000**	**0.462**	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**

log p(image) = -1.6740

p(image) = 0.18750

> Sampling builds the image pixel by pixel. Density evaluation uses an image that is already known.

Toy data: 32 binary 3 × 3 images with stripes. Conditional probabilities come from counts.

Autoregressive Models: MLP

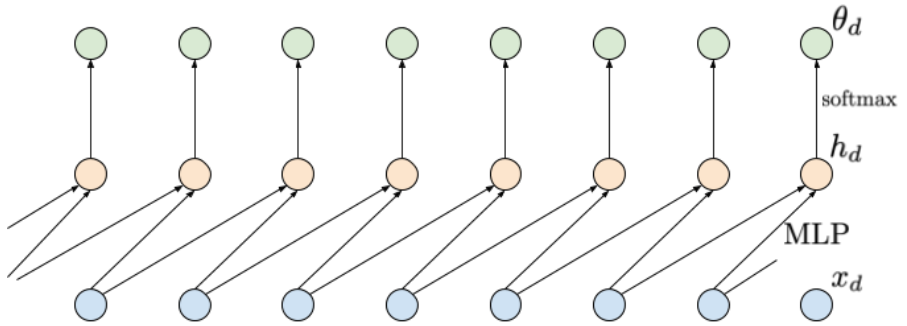
For large j, the conditional $p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{1:j-1})$ becomes intractable as the history $\mathbf{x}_{1:j-1}$ grows variable-length.

Markov Assumption

$$p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{1:j-1}) = p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{j-d:j-1}), \quad d \text{ is a fixed parameter.}$$

Example

- $d = 2$
- $x_j \in \{0, \dots, 255\}$
- $\mathbf{h}_j = \text{MLP}_{\boldsymbol{\theta}}(x_{j-1}, x_{j-2})$
- $\boldsymbol{\pi}_j = \text{softmax}(\mathbf{h}_j)$
- $p_{\boldsymbol{\theta}}(x_j|x_{j-1}, x_{j-2}) = \text{Cat}(\boldsymbol{\pi}_j)$



Can we also model continuous-valued data, not just the discrete case?

Image credit: https://jmtomczak.github.io/blog/2/2_ARM.html

Autoregressive Models: LLM

$$p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{1:j-1}) = p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{j-d:j-1}), \quad d \text{ is the context window.}$$

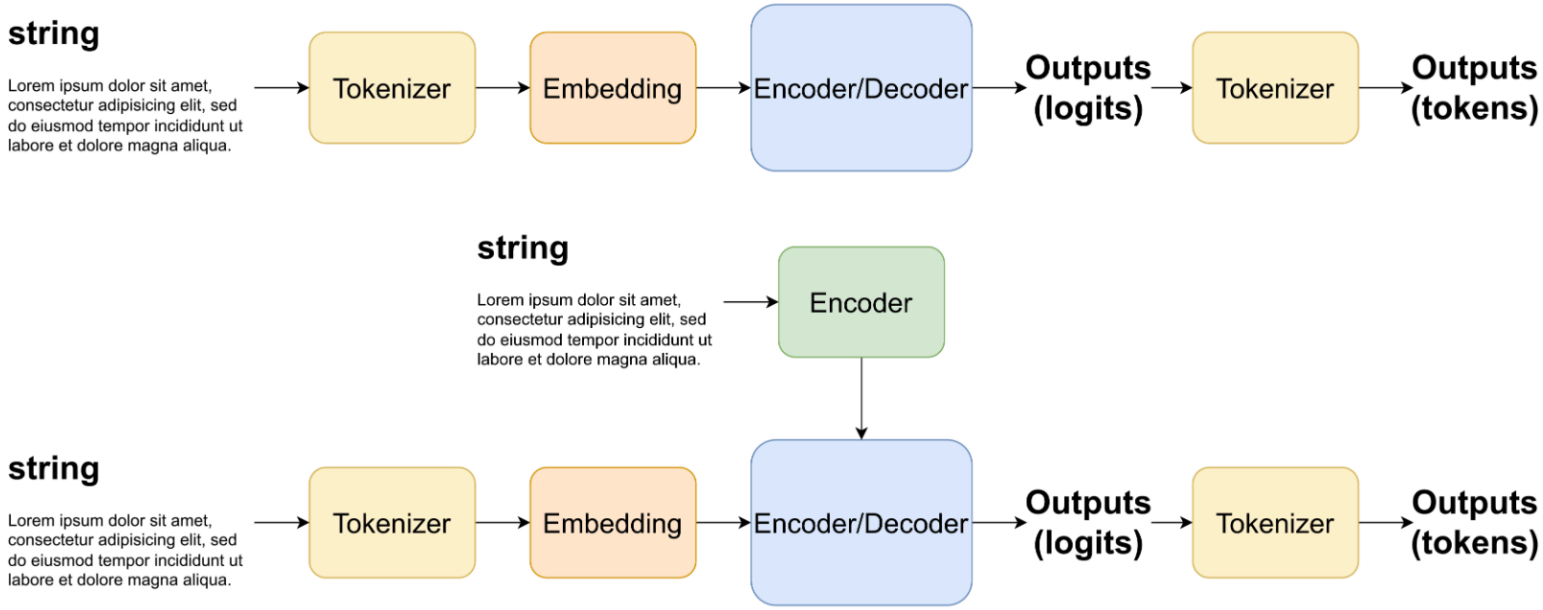


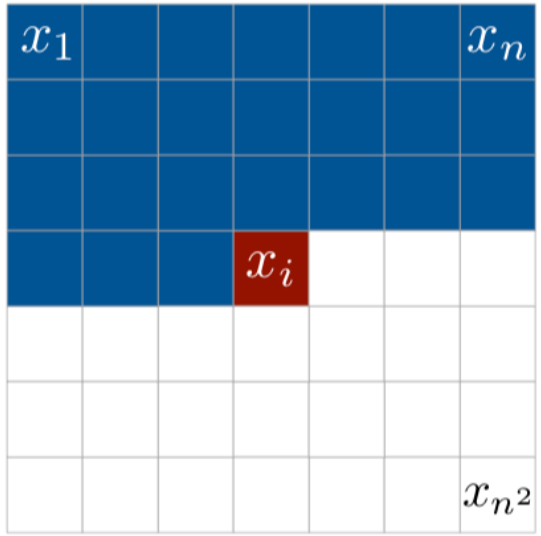
Image credit: https://jmtomczak.github.io/blog/20/20_llms.html

Autoregressive Models for Images

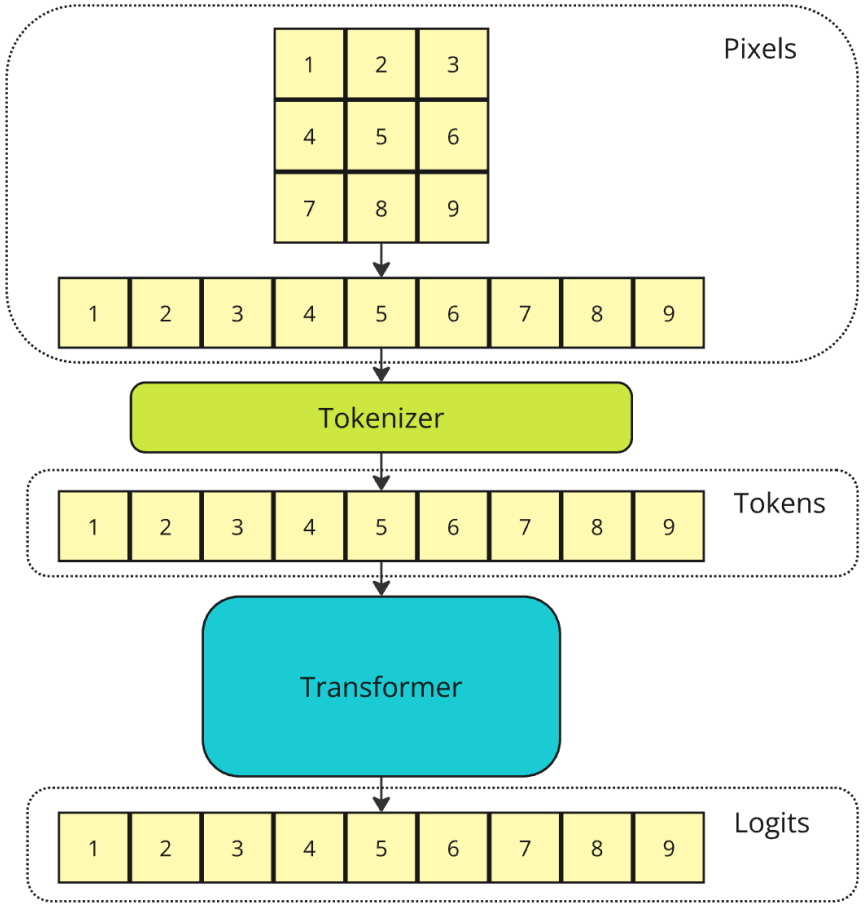
How do we model the distribution $p_{\text{data}}(\mathbf{x})$ of natural images?

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{j=1}^{\text{width}\times\text{height}} p_{\boldsymbol{\theta}}(x_j | \mathbf{x}_{1:j-1})$$

- A pixel ordering must be selected; the raster scan is a standard choice.
- RGB channel dependencies can be modeled explicitly as well.



Autoregressive Models: ImageGPT



Chen M. et al. Generative Pretraining from Pixels, 2020

Summary

- Density estimation learns a distribution from data; sampling and density evaluation use the fitted model.
- Model families: autoregressive models, normalizing flows, VAEs, GANs, and diffusion models.
- The divergence minimization framework offers a principled way to learn distributions that match the data.
- Minimizing the forward KL divergence is equivalent to maximum likelihood estimation.
- Autoregressive models factorize the joint distribution; density evaluation multiplies the conditionals.
- ImageGPT applies a transformer to raster-ordered image pixels; autoregressive sampling is sequential.